

Progmune

AI 代码信任决策引擎

让 AI 生成的每一行代码，在进入生产环境之前，都经过可解释、可审计、可追溯的验证。本文档面向两类读者：想快速理解的初学者，与需要专业判断的投资人。



版本 v1.0 · 2026-08



引擎版本 v3.2.0



13/13 安全类别覆盖



开源 MIT



npm install progmune-runtime

目录 · CONTENTS

- | | |
|-------------------------------|----------------------------------|
| 1. 01 三十秒理解 Progmune | 7. 07 实测案例：PrintLab 治理全过程 |
| 2. 02 名词解释（先读这个） | 8. 08 竞争格局 |
| 3. 03 项目背景：AI 代码的信任危机 | 9. 09 市场机会与商业模式 |
| 4. 04 框架逻辑：为什么这样做能解决问题 | 10. 10 风险分析（客观） |
| 5. 05 技术方法：具体怎么做 | 11. 11 路线图 |
| 6. 06 产品与能力现状 | 12. 12 给投资人的总结论 |

01

三十秒理解 Progmune

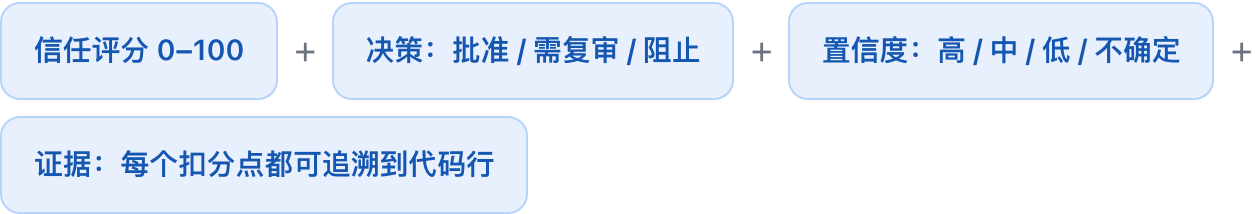
一句话：**Progmune** 不生成代码，它给 AI 生成的代码做"上岗体检"，并给出一个明确结论——能不能部署。

一个类比：飞机在起飞前有完整的检查单——油量、襟翼、液压、仪表。AI 现在以 10 倍速度"造飞机"，但绝大多数企业让这些飞机**没有检查就直接起飞**。Progmune 就是那张检查单：不是替飞行员开飞机，而是确保每一架飞机在起飞前都通过了必检项。

Progmune 检查的不是"代码风格好不好"，而是四类传统工具看不见的问题：

问题类型	通俗解释	例子
符号幻觉	AI 调用了项目里根本不存在的函数	代码里写 <code>getUserByPhone()</code> ，但项目只有 <code>getUserByEmail()</code>
类型漂移	参数数量或类型对不上	函数要 3 个参数，AI 只传了 2 个
数据流错误	变量没定义就使用	用了 <code>userName</code> 但它从未被赋值
协议违规 ★ 核心	关键步骤顺序错了或漏了	没验证密码就签发令牌；支付完成但没验证签名

检查结果不是一份问题清单，而是一个**决策**：



为什么企业需要它：企业真正关心的问题是"这段 AI 代码能不能上线？"——不是"它的分数是 58 还是 61"。Progmune 把这个问题的答案做成了一条 CI/CD 命令：`BLOCKED` 就阻止合并，`APPROVED` 就放行。

02

名词解释（建议先读）

本文档用到的所有专业术语，按"先易后难"排列。读完这一节，后面不会再被术语挡住。

基础层

大语言模型 (LLM) Large Language Model

就是 ChatGPT、Claude、DeepSeek 这类 AI 的"大脑"。它会"猜"最可能正确的下一个词——写代码对它来说就是"猜下一行"。

AI 生成代码 AI-Generated Code

由 AI 工具（如 GitHub Copilot、Cursor）写出或协助写出的代码。它的特点是：语法通常正确，但语义可能错。

幻觉 Hallucination

AI 一本正经地输出不存在的东西。写代码时表现为：调用不存在的函数、用不存在的参数——而且不报错。

开源 Open Source / MIT

代码完全公开，任何人可免费使用、修改、商用。Progmune 采用 MIT 协议，与 GitLab、VS Code 同类。

概念层

中间表示 IR Intermediate Representation

把项目源码解析成一份"函数清单"（名字、参数、类型、调用关系）。这份清单从真实代码提取，不依赖 AI 的记忆——是系统的"唯一真相来源"。

协议 Protocol

事情必须按特定顺序做的规则。例如：登录 = 先验证密码 → 再签发令牌 → 再创建会话。顺序错了就是安全漏洞。

状态机 State Machine

一种数学工具，用来描述"现在处于什么状态、允许做什么操作"。交通信号灯就是状态机：红灯不能变黄灯，绿灯不能直接跳红灯。

SVL 四层验证 Semantic Validation Levels

Progmune 的检查体系，共四层：SVL-1 函数存在性 → SVL-2 类型正确性 → SVL-3 数据流 → SVL-4 协议顺序。层层递进，越往后越难、越关键。

SSG State-Guided Generation

Progmune 的核心技术：用状态机"引导"AI 生成代码——AI 每提出一步，状态机就检查这一步是否合法，不合法就纠正。

免疫记忆 / 抗体 Immune Memory / Antibody

比喻：系统把每次发现的错误模式记下来。同类错误再出现时，直接调用已知解法（像人得了一次水痘后终身免疫）。

产品层

信任评分 Trust Score

0-100 的量化分数，衡量一段 AI 代码的可信程度。由四个维度加权得出（见第 4 章）。

部署决策 Decision

APPROVED（批准）/ NEEDS_REVIEW（需人工复审）/ BLOCKED（阻止）。这是产品的最终输出——CI/CD 管道直接按它行动。

置信度 Confidence

这个评分本身有多可靠。Progmune 的原则是“诚实”：对没能力检查的语言（如 Go），明确说“置信度低”，不假装 100%。

PLSB Protocol Lifecycle Security Benchmark

协议生命周期安全基准——13 类协议安全弱点的分类体系（如资源泄漏、认证绕过、会话劫持）。Progmune 是首个覆盖全部 13 类的验证引擎。

MCP Model Context Protocol

AI 客户端（如 Claude Code）与工具服务器之间的标准接口——相当于 AI 世界的“USB 接口”。Progmune 通过它接入各种 AI 编程工具。

CI/CD Continuous Integration/Deployment

软件开发中的自动化流水线：代码提交后自动测试、自动部署。Progmune 的决策就嵌入在这里——不合格的代码自动被拦下。

SAST / 静态分析 Static Application Security Testing

逐行扫描代码找已知漏洞模式的传统工具（CodeQL、Semgrep 属于此类）。它们的盲区：看不见跨函数的调用顺序错误。

tRPC / Express Web Frameworks

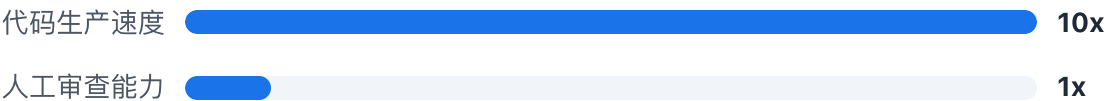
两个流行的 Web 后端框架。可以理解为“搭建网站后端的脚手架”——Progmune 针对这些框架做专项适配。

阅读提示：如果后面再遇到不懂的词，回到这里。核心记住三件事就够了：① **IR = 真相清单**（真实代码里有什么函数）；② **协议 = 顺序规则**（必须先做什么再做什么）；③ **决策 = 最终答案**（能上线 / 不能上线）。

3.1 时代背景：代码生产速度暴涨，验证能力原地踏步



AI 编码工具让代码的生产速度提升了约 10 倍，但验证环节几乎没有变化——仍然靠人工审查和传统测试。GitClear 2024 报告显示，AI 代码的变更回滚率比传统代码高 14%。这形成了一个清晰的剪刀差：



3.2 问题本质：AI 写代码的原理决定了它"看不见"错误

大语言模型生成代码的方式是**统计预测**——根据训练数据猜"下一个最可能的词"。这一观点的代表性论述来自 Subbarao Kambhampati 等的立场论文《Stop Anthropomorphizing Intermediate Tokens as Reasoning/Thinking Traces!》（arXiv:2505.22285）及其 ICML 2026 关于 verifier 的主题演讲：LLM 输出的"逐步推理"是统计表演，中间 token 与真实推理之间没有可靠的因果对应。

这意味着：AI 写出"看似合理的代码"和"实际正确的代码"是两件事。它会一本正经地调用不存在的函数、跳过关键的认证步骤——而且完全静默，不报错。

3.3 现有方案为什么不够

方案	原理	根本缺陷
Linters / SAST CodeQL, Semgrep, SonarQube	代码写完后逐行扫描 已知漏洞模式	只看到"单行"，看不到"跨函数调用顺序"——协议违规是它们的系统性盲区

RAG 上下文注入	把项目信息塞进 AI 的提示词，降低幻觉	降低但不消除幻觉；AI 仍然是唯一仲裁者；无形式化保证
提示词工程	精心设计指令引导 AI	零合规保证；AI 版本一升级就失效
人工审查	人眼逐行检查	不可扩展——生产速度快 10 倍，人不会快 10 倍
单元测试	写测试用例验证行为	测试覆盖有限；测试本身也常由 AI 生成（自证循环）

关键洞察：以上所有方案有一个共同点——它们都是**"事后补救"**。错误已经写进代码，再去发现它。而真正的问题发生在代码生成的那一刻。这就是 Progmine 切入的角度：**在生成环节植入确定性验证。**

3.4 为什么是现在

- **临界点已到：**AI 编码从实验走向企业标配，但没有任何"编译器级"的验证机制保护它
- **监管在收紧：**金融（SOX）、医疗（HIPAA）等受监管行业开始要求 AI 代码可审计、可追溯
- **事故在积累：**AI 生成代码导致的线上事故（支付漏洞、认证绕过）开始进入公众视野
- **技术窗口：**协议状态机 + 中间表示的组合方案在学术界和工业界都还没有成熟竞争者

04

框架逻辑：为什么这样做能解决问题

Progmine 的全部设计源于一个第一性原理：**不信任 AI 说的，只验证程序实际做的。**

4.1 核心思想：把 AI 从"预言机"降级为"受约束的提议器"

传统模式下，AI 是系统的中心：它写什么，什么就是代码。Progmine 翻转了这个关系：



AI 仍然负责"创意"（写什么代码），但**不再拥有"裁决权"**（代码合不合格）。裁决权交给一个确定性的验证系统——同一段代码、同一套规则，永远得到同一个结果。

4.2 三大支柱

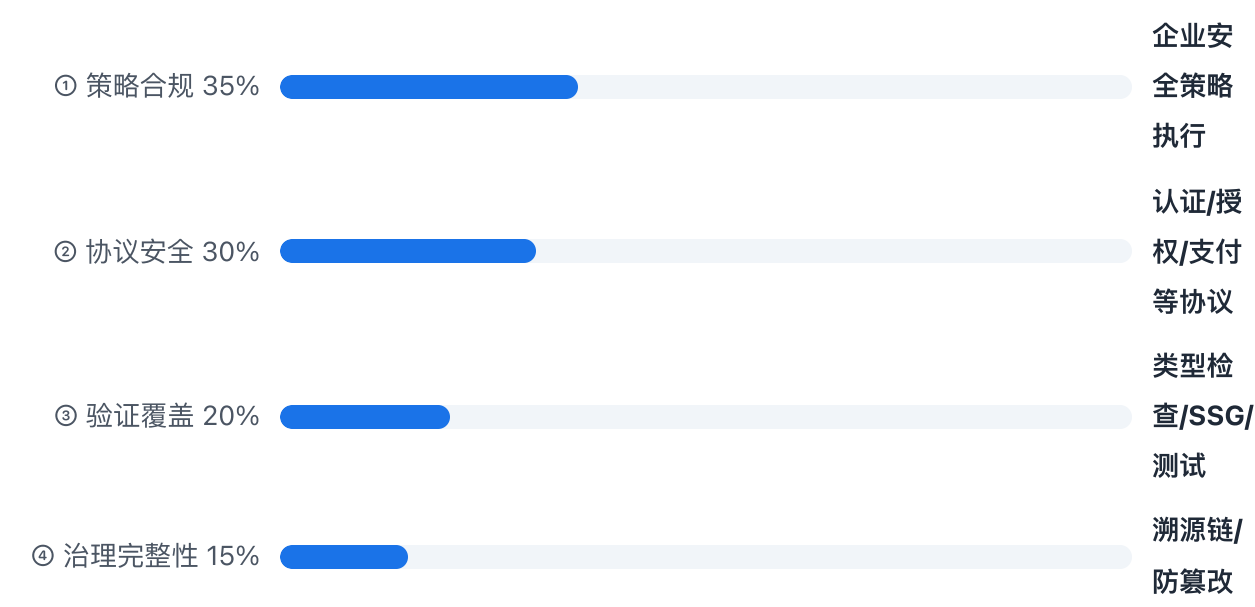
支柱	通俗解释	为什么不可或缺
① 程序真相层 (IR)	把项目真实存在的函数、类型、调用关系解析成一份清单	AI 的"记忆"是概率性的，IR 是确定性的。没有它，验证无从谈起——它是所有检查的事实基础
② 协议状态机 (SSG)	把"必须按顺序做的事"建模成状态机，检查每一步是否合法	这是与传统 SAST 的本质区别：传统工具看"单行代码"，Progmune 看"调用顺序"。认证前签发令牌这种漏洞只有顺序检查能发现
③ 免疫记忆	每次拦截的错误都记入语料库，形成可复用的"抗体"	让系统越用越强：同类错误第二次出现时直接调用已知解法，甚至跳过 AI 重试——形成数据飞轮

4.3 四层验证（SVL）：从便宜到昂贵，层层递进

层级	检查什么	拦截的问题	检查成本
SVL-1 符号存在性	调用的函数在项目里真实存在吗？	幻觉函数名	极低（查表）
SVL-2 类型完整性	参数数量和类型与签名匹配吗？	类型漂移	低
SVL-3 数据流正确性	变量先声明后使用了吗？	未定义变量、循环引用	中
SVL-4 协议合法性 ★	调用顺序符合协议状态机吗？	认证绕过、支付篡改、会话劫持	较高（状态机 + BFS 搜索）

4.4 决策模型：四个维度加权出一个"能不能上线"

Progmune 的最终输出不是问题清单，而是四维加权后的一个决策：

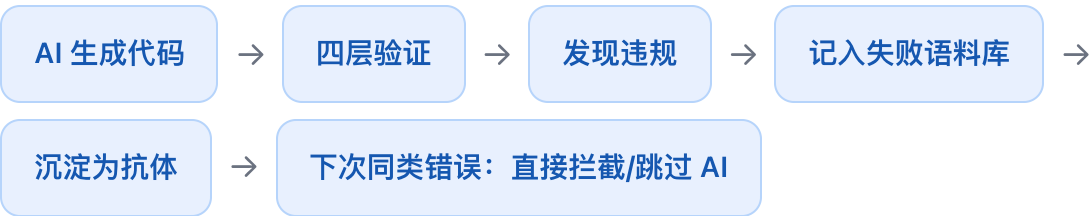


判定规则（简单明确）：

条件	决策
存在严重违规（critical）	BLOCKED —— 不管分数多高，一律阻止
总分 ≥ 80	APPROVED —— 批准部署
60-79	NEEDS_REVIEW —— 需人工复审
< 60	BLOCKED —— 阻止

设计哲学（也是产品哲学）：Decision > Score。企业关心"能不能上线"，不关心"58 分还是 61 分"。所以决策是产品，分数只是支撑决策的证据。同时坚持"诚实原则"：检查不到的能力边界，用置信度明确标注，绝不假装 100% 覆盖。

4.5 学习闭环：数据飞轮



每次拦截都让系统更强大。ACL-4 级抗体（同模式出现 ≥ 10 次且跨 ≥ 5 个场景）能实现零 LLM 调用直接修复——既省钱又更可靠。这是 Progmune 网络效应的技术基础。

05 技术方法：具体怎么做

本章说明技术实现，尽量用类比解释。技术读者可关注括号内的模块名。

5.1 IR 提取：建立"真相清单"

用 TypeScript 编译器 API（`ts-morph`）解析项目源码，提取：函数声明、箭头函数、参数类型、返回类型、JSDoc 注解（`@protocol` 标记）、调用图、外部依赖（约 40 个常用库函数签名）。Python 项目也有对应提取器。

```
源码: function createOrder(userId: number, items: Item[]): Order
      ↓ 解析
IR: { name: "createOrder", params: [{name:"userId", type:"number"}...],
      returnType: "Order", calls: ["verifyPayment", "insertRecord"] }
```

5.2 协议状态机：把"顺序规则"变成可执行检查

每个关键协议被建模为一个状态机。以登录为例：

```
规则 (protocols.json, 共 148 条, 覆盖 21 个命名空间) :
  verify_password:  UNAUTHENTICATED → PASSWORD_VERIFIED
  generate_jwt:     PASSWORD_VERIFIED → TOKEN_ISSUED
  create_session:  TOKEN_ISSUED      → SESSION_ACTIVE
  logout:          SESSION_ACTIVE    → UNAUTHENTICATED

AI 生成: create_session(...) ← 当前状态 UNAUTHENTICATED
验证器:  × 拒绝! create_session 需要前置状态 TOKEN_ISSUED
修复器:  BFS 搜索最短修复路径 → [verify_password, generate_jwt]
        自动插入这两个调用, 重新验证 ✓
```

修复是确定性的：BFS 图搜索保证找到最短修复路径，无需再次调用 AI。

5.3 检测器与防护规则：静态模式的补充

- **22 个协议检测器**：TLS 握手、SSH 连接、HTTP 请求、支付处理、会话管理、文件上传、注册流程等
- **26 条防护规则 (safeguard)**：触发式检查——"如果出现了 X，就必须有 Y"。例：注册函数必须调用安全哈希 (bcrypt/argon2)；管理操作必须检查角色；tRPC mutation 必须有 `.input()` 校验 schema
- **15 条源码级检测规则 (Python, 2026-08 上线)**：SQL 注入 (f-string/%.format/拼接, 参数化写法正确放行)、SSRF (限定接收者的 HTTP 抓取 + request 污点追踪)、路径穿越 (文件 sink + 污点)、XSS (跨文件: 模板 `{{ var|safe }}` × 污点上下文绑定)、SSTI、XXE (解析器配置 + 污点双信号)、命令注入 (仅动态命令参数)、反序列化、CSRF 双形态 (`@csrf_exempt` / GET 状态变更)、eval 代码执行、硬编码 JWT 密钥 (含跨模块常量)、cookie 授权、可预测重置令牌
- **提取器标记架构**：IR 提取器做源码级分析 (污点追踪、import 解析、限定调用链、模板扫描、跨模块常量表) → 合成标记 → 规则消费。零管道改动、完全可审计，已同步移植到 TS (ts-morph) 提取器

5.4 API 语义映射：让检查认识"陌生函数"

真实项目使用的函数名千变万化 (`createSessionToken`、`signSession`、`jwt.sign`)。系统用三层策略把陌生函数映射到协议规则：① 本地前缀/别名表 (含项目自定义 `.progmune_aliases.json`) → ② 单词切分匹配 → ③ LLM 批量语义分类 (带磁盘缓存, 重复运行零 LLM 成本)。

5.5 双路径架构：生成时拦截 vs 事后检查

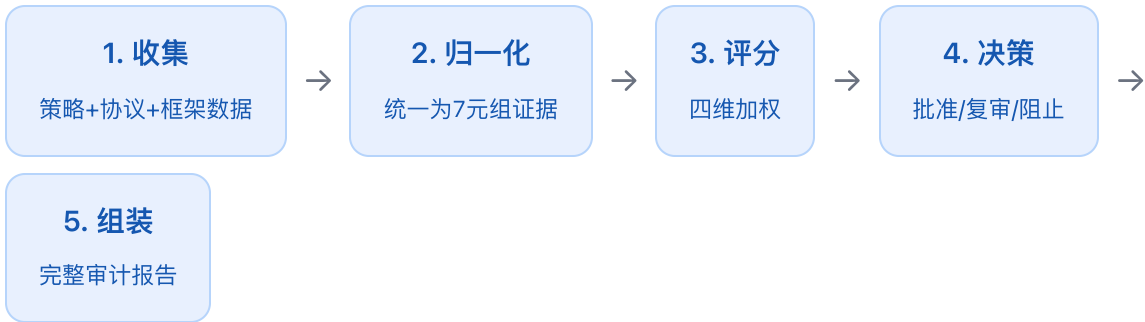
Progmune 用两条互补路径覆盖两类代码来源——这是理解产品的关键，也是最容易被误读的地方：

生成路径（生成时拦截）		验证路径（事后检查）
覆盖场景	通过 <code>progmune_generate</code> 生成的代码	任何来源的代码（Copilot、Cursor、人工）

机制	生成循环内 8 道验证关卡：JSON 解析 → 结构预检 → 符号纠正 → 类型修正 → 数据流 → 协议状态机 → BFS 确定性修复 → 语义合约。违规代码从不写入文件	信任引擎五阶段流水线（收集→评分→决策→组装），四维加权输出部署决策
错误处理时机	创建时刻——错误从未存在	事后——文件已存在
成本曲线	零——违规没有落地	发现越晚越贵

核心产品论：在生成时刻验证，而非事后补救。LLM 的输出只是提议，状态机才是裁判——LLM 可以被劝服，状态机不会。

5.6 信任引擎：五阶段流水线



每条违规都携带**7 元组证据**：规则 ID、文件、函数、代码证据、为什么违规、如何修复、关联策略。这是"可解释性"的技术基础——每一分的扣减都能追溯。

5.6 框架适配器：让检查理解具体框架

不同框架（Express、tRPC、NestJS、Next.js）的认证、校验、限流写法各不相同。适配器逐框架解析：Express 的中间件链、tRPC 的 procedure 声明、框架版本差异（如 Next.js 16 用 `proxy.ts` 而非 `middleware.ts`）。

5.7 基准测试：用数据证明能力

基准	方法	结果
盲测基准（TypeScript, 100 项目）	90 个合成风格变体 + 10 个模型变体项目、795 条 gold finding、严格定位匹配	精确率 100% · 召回率 98.5% （有效口径 100%）· 事实性误报 0
盲测基准（Python, 90 项目）	90 个合成风格变体、729 条 gold finding	精确率 100% · 召回率 100% · 事实性误报 0
真实应用验证（PyGoat）	OWASP 故意脆弱 Django 应用、232 条检测逐条人工核实	标记精确率 100% （67 真阳性 / 0 误报）；覆盖 14 个漏洞类别
良构应用（django/fastapi realworld、django-unicorn）	176 条检测	0 条误报真阳性；3 条框架内部边界 FP（已定性归档）
黄金基准（C 语言）	curl/libssh/nginx/openssl 等真实 CVE 案例	F1 16.5% —— 研究阶段，诚实披露
PLSB v1.0	13 类协议安全弱点、39 个手工验证缺陷案例	覆盖 13/13 全类别 （业界唯一）

为什么披露 C 语言的低分？因为"诚实"是产品原则。一个只报告好消息的基准没有参考价值。TS 与 Python 已生产可用、C 为研究课题——能力边界清清楚楚。所有基准均有 gold 标注文件入库，可复现。

6.1 产品形态

形态	说明	状态
MCP 服务器	19 个工具： <code>progmune_generate</code> （受约束代码生成）、 <code>progmune_trust_check</code> （信任决策）、 <code>progmune_score</code> （生成前预检）、 <code>progmune_certify</code> （证书签发）等，接入 Claude Code 等 AI 客户端	✅ 可用
SDK	一行调用： <code>verify(file) → BLOCK/WARN/ALLOW</code>	✅ 可用
CLI	<code>npm run trust</code> 、 <code>npm run governance</code> —— 命令行信任检查与治理报告（终端/JSON/Markdown/HTML 四种格式）	✅ 可用
GitHub Action	PR 合并前自动门禁	✅ 可用
Trust API	<code>POST /trust/check</code> 机器间接口	✅ 可用
SaaS 仪表盘	多项目可视化管理	📅 规划中（等 PoC 验证后）

6.2 语言与框架覆盖（诚实版）

能力	状态	说明
TypeScript / JavaScript	✅ 生产	盲测 100 项目：精确率 100%、召回率 98.5%

Python	✅ 生产	盲测 90 项目：精确率/召回率双 100%；15 条源码级检测规则；PyGoat 真实验证 67 TP / 0 FP
C	⚠️ 研究	F1 16.5%，瓶颈已定位（规则覆盖），L3 实验已终止
Go / Java	❌ 未支持	路线图内
框架适配	2/13	Express ✅、tRPC ✅、NestJS 部分；Next.js 版本感知已实现；Django/FastAPI 经 Python 源码级检测间接覆盖

6.3 知识库规模

148 协议规则	21 协议命名空间全覆盖	22+26+15 检测器 + 防护 + 源码级规则
39 黄金缺陷案例（人工验证）	190 盲测项目（TS 100 + Python 90）	4 真实应用验证仓库

07 实测案例：PrintLab 治理全过程

PrintLab 是一个真实的 3D 打印电商项目（Express + tRPC + React，59 个后端文件）。用 Progmune 对它做完整治理，结果如下——这是产品能力最直接的证据。

7.1 第一轮：发现 46 个问题（其中大量误报）

 **BLOCKED · 信任评分 44**



46 个违规：1 critical + 1 high + 45 medium。但其中 43 个经分析是状态机规则粒度过细导致的误报。

7.2 调优：状态机规则修正

逐个分析误报根因并修正规则——例如：通知发送函数在真实代码中"组合+发送"一步完成，但状态机要求先显式"组合"；一次性文件读取 API（`readFileSync`）内部自带打开/关闭，但状态机要求显式 `open`。共修正 8 类状态机规则 + 2 处别名误配。

这个过程本身就是产品价值的一部分：状态机规则不是死代码，而是像"公司制度"一样持续迭代。每次误报分析都在提升规则的精确度——这正是"越用越准"的飞轮在真实项目上的第一次完整运转。

7.3 真实发现（3 个）

严重度	位置	问题	修复
HIGH	优化商品路由	管理员 mutation 批量删除/更新商品，无输入校验 schema	添加 <code>.input(z.void())</code> 显式声明合约
MEDIUM	认证工具	JWT 验证未指定算法白名单——"none"算法混淆攻击风险（CVE-2022-23540 模式）	添加 <code>algorithms: ["HS256"]</code>
MEDIUM	会话 SDK	第二处 JWT 验证同样缺少算法白名单	同上

7.4 最终结果



APPROVED · 信任评分 87 · 置信度 HIGH · 0 违规

46 个问题 → 0 个问题；44 分 BLOCKED → 87 分 APPROVED。三个真实安全问题在代码进入生产前被修复。

治理报告生成为专业 HTML 文档（含通俗解释），可直接用于向 CTO/合规团队/投资人展示。

08 竞争格局

8.1 定位：不与 Copilot/Cursor 竞争，做它们的安全底盘

AI 编码市场已由 GitHub Copilot、Cursor 等巨头占据——Progmune 不与他们竞争。Progmune 处于另一层：验证层。类比：汽车厂商竞争造车，Progmune 做的是碰撞测试——所有车厂都需要，但没人会把它当作"竞争对手"。

8.2 七维对比

能力维度	传统 SAST CodeQL/Semgrep	Prompt 安全 Guardrails AI 等	Progmune
介入生成过程	× 事后扫描	△ 输入/输出过滤	✓ 生成时验证
协议顺序检测（跨函数）	×	×	✓ SSG 状态机
形式化保证	×	×	✓ SVL 1-4
可解释证据（每分可追溯）	部分	×	✓ 7 元组证据
模型无关	✓	× 绑定模型	✓ 支持任何 LLM
越用越强（学习能力）	×	×	✓ 免疫记忆飞轮
输出部署决策	× 输出告警	×	✓ APPROVED/BLOCKED

8.3 竞争壁垒（客观评估）

壁垒	性质	客观评估
IR 真相层	技术架构	从源码确定性提取，不可伪造。但实现难度中等——大厂有追赶能力
SVL 分类法	标准制定	类似 PCI-DSS 之于支付安全。标准一旦被行业采纳，先发优势巨大——但采纳与否未验证
免疫记忆数据	数据飞轮	最有价值的壁垒：每次拦截都积累数据。但需要足够的使用量才能形成规模效应
协议规则库（148 条）	领域知识	从真实 CVE 和缺陷案例提炼，难以从零复制。但可被持续追赶
开源社区先发	生态	MIT 协议 + MCP 原生集成，建立开发者心智——目前尚在早期

09

市场机会与商业模式

9.1 市场规模

\$25B+ AI 代码生成市场（2030E）	\$9B+ DevSecOps 市场（2024）	\$3–5B AI 安全层近期可寻址市场
---	--	--

核心逻辑：AI 代码生成市场每增长 1%，对验证的需求就增长 1%——这是一个与 AI 编码同步增长的伴生市场。而且与 AI 编码不同，验证层目前没有巨头。

9.2 目标客户

客户画像	痛点	Progmune 提供
大规模部署 Copilot/Cursor 的企业	AI 代码进生产无人把关	CI/CD 自动门禁 + 审计报告
金融/医疗等受监管行业	SOX/HIPAA 要求 AI 代码可审计	7 元组证据链 + 治理报告
平台工程团队	多工具 AI 编码各自为政	统一验证层 + 集中策略
AI 原生初创公司	Agent 直推生产无安全网	部署前最后一道确定性防线

9.3 商业模式：开源内核 + 企业增值

层级	定价	内容	定位
开源版 (MIT)	永久免费	SVL 四层验证、TS/Python IR、MCP 集成、本地语料库	建立生态与信任
企业版	按席位/年	CI/CD 门禁集成、团队仪表盘、高性能验证器 (Rust/WASM) 、私有 Hub、SLA	营收主力
免疫网络	按组织/年	全球失败指纹联邦、行业协议模板、威胁情报推送	网络效应锁定

此模式已被 GitLab、Databricks 验证：开源建立信任和分发，企业版解决付费问题。

10

风险分析（客观）

投资人需要知道风险，本节不粉饰。

风险	等级	内容	缓解措施
----	----	----	------

市场教育成本	高	多数企业尚未意识到"AI 代码需要验证"这个问题本身	公开缺陷基准库、从受监管行业切入、开源社区自然教育
大厂竞争	中	GitHub/Microsoft 可能内置类似功能	聚焦"验证层"差异化；大厂内置方案通常浅层；开源先发
验证的边界性	中	TS 与 Python 已生产可用；协议规则不可能覆盖所有场景——存在漏检的可能	诚实披露置信度；数据飞轮持续补规则；规则来自真实 CVE 案例
LLM 依赖	低	代码生成环节依赖底层模型能力	架构上模型无关；验证器本身不依赖任何 LLM；模型越强产品越有价值
商业化节奏	中	开源到付费的转化路径需要时间验证；尚无企业 PoC	GitLab/Databricks 模式已成熟；优先做 2-3 个企业 PoC 验证 PMF
核心假说未验证	中	"每个新代码库让所有验证更强"的知识网络假说尚无实证	以 Phase 2 的 PoC 数据为准；假说不成立也不影响单点验证的产品价值

最需要投资人关注的风险：市场教育成本。Progmune 卖的不是"更快的工具"，而是"信任"——这类产品的销售周期天然更长。对冲策略是：用公开基准数据（PLSB、盲测结果）让产品"自证"，而不是靠销售话术。

11 路线图

阶段	时间	目标	关键交付
当前	2026-08	Trust Foundation +	信任引擎✅、策略引擎✅、PLSB 13/13✅、tRPC/Express 适配✅、PrintLab 案例✅、

语言扩展完成

Python 生产级✅（盲测双 100%、PyGoat 真实验证 0 误报）✅、15 条源码级检测规则✅、TS/Python 盲测 precision 100%✅

Phase 1 收官	2026 Q3	可信度校准	已知风险项目上的 Score 校准、真实项目验收、Trust Report PDF
Phase 2	2026 Q4	企业验证	企业 PoC × 2-3、Evolution 追踪、CI/CD 门禁集成、良构应用框架内部 FP 收尾（类属性建模）
Phase 3	2027+	平台化	Go/Java、SaaS 仪表盘、公开基准排行榜、全球免疫网络、协议模板市场

12

给投资人的总结论

为什么值得投

1. 真问题 + 真窗口：76% 企业已用 AI 编码，验证层是明确的基础设施缺口，且没有巨头占位
2. 差异化足够深：协议顺序验证是传统 SAST 的系统性盲区——这是"品类级"差异，不是"功能级"差异
3. 数据飞轮成立：每次拦截都沉淀为抗体，使用量即壁垒（与 Salesforce 的网络效应同构）
4. 开源已验证：19 个工具、148 条规则、85.2% F1、真实项目案例（PrintLab 从 BLOCKED 到 APPROVED 的完整过程可复现）

为什么需要谨慎

1. 企业 PoC 为零——PMF 未验证，商业化路径尚需数据
2. 语言覆盖有限（TS/Python 生产可用，Go/Java 未支持）
3. "信任"类产品教育成本高，销售周期长
4. 大厂随时可能跟进（但架构先发 + 数据积累构成时间壁垒）



建议的验证路径

与其争论技术细节，不如用 3 个月做一件事：在 2-3 家真实企业跑 PoC——让企业用自己的 AI 代码、自己的事故案例，看到 Progmune 拦住真实问题的过程。PMF 一旦验证，后续的资金故事自然成立。

Progmune — AI Trust Decision Engine · 本文档 v1.0 · 2026-08

github.com/shenlian19831109/progmune-runtime · npm install progmune-runtime · MIT License